

Data Analysis with Python 2024–2025

Parte 3: Mais NumPy

Caderno de Exercícios

Tradução e Adaptação: University of Helsinki — MOOC.fi

Nota Importante

Este material é uma tradução e adaptação baseada no curso *Data Analysis with Python 2024-2025*, oferecido pela **The University of Helsinki MOOC Center**.

Conteúdo

1 Exercícios - Capítulo 3

Exercício 3.11 – para tons de cinza (*to grayscale*)

Sumário

- **Parte 1:** Escreva uma função `to_grayscale` que receba uma imagem RGB (array tridimensional) e retorne uma imagem bidimensional em tons de cinza. A conversão para tons de cinza deve ser feita através de uma soma ponderada dos valores vermelho, verde e azul, e usar isso como o valor de cinza. O primeiro eixo é x , o segundo é y , e o terceiro são os componentes de cor (vermelho, verde, azul). Use os pesos 0.2126, 0.7152 e 0.0722 para vermelho, verde e azul, respectivamente. Esses pesos existem porque o olho humano é mais sensível à cor verde e menos sensível à cor azul. Na função `main`, você pode, por exemplo, usar a imagem fornecida `src/painting.png`. Exiba a imagem em tons de cinza com a função `plt.imshow`. Pode ser necessário chamar a função `plt.gray` para definir a paleta de cores (colormap) como cinza. (Veja `help(plt.colormaps)` para mais informações sobre colormaps).
- **Parte 2:** Escreva as funções `to_red`, `to_green` e `to_blue` que recebem um array tridimensional como parâmetro e retornam um array tridimensional. Por exemplo, a função `to_red` deve zerar os componentes de cor verde e azul e retornar o resultado. Na função `main`, crie uma figura com três subfiguras: a de cima deve ser a imagem vermelha, a do meio a imagem verde e a de baixo a imagem azul.

Exercício 3.12 – desvanecimento radial (*radial fade*)

Sumário

- **Objetivo:** Faça um programa que aplique um desvanecimento (*fading*) em uma imagem como visto anteriormente, mas agora não na direção horizontal, e sim na direção radial. À medida que nos afastamos do centro da imagem, os pixels desvanecem para o preto.
- **Parte 1:** Escreva a função `center` que retorna o par de coordenadas (`center_y`, `center_x`) do centro da imagem. Observe que essas coordenadas podem não ser inteiras. A função deve funcionar tanto para imagens bidimensionais quanto para tridimensionais (ou seja, imagens em tons de cinza e coloridas). Escreva também a função `radial_distance` que retorna para uma imagem com largura w e altura h um array com forma (h, w) , onde o número no índice (i, j) informa a distância euclidiana do ponto (i, j) até o centro da imagem.
- **Parte 2:** Crie a função `scale(a, tmin=0.0, tmax=1.0)` que retorna uma cópia do array `a` com seus elementos escalados para estarem no intervalo $[tmin, tmax]$. Usando as funções `radial_distance` e `scale`, escreva a função `radial_mask` que recebe uma imagem como parâmetro e retorna um array com a mesma altura e largura preenchido com valores entre 0.0 e 1.0. Faça isso usando a função `scale`. Para fazer com que os valores do array resultante perto do centro do array fiquem próximos a 1 e mais perto das bordas fiquem próximos a 0, subtraia o array anterior de 1. Escreva também a função `radial_fade` que retorna a imagem multiplicada por sua máscara radial. Teste suas funções na função `main`, que deve criar, usando o matplotlib, uma figura que tenha três subfiguras empilhadas verticalmente. No topo, a imagem original (`painting.png`), no meio, a máscara, e na parte inferior, a imagem com desvanecimento.

Créditos: Este conteúdo foi originalmente criado pela *The University of Helsinki MOOC Center* para o curso *Data Analysis with Python*.